08 · Citizen record

One citizen's full picture for the health worker, for a chosen time window.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The record page shows: open flags with review controls; the latest value of each vital against the person's own baseline; their health profile; a chart per vital; every submission (flagged ones with a "vs their baseline" tag and an expandable "Why it was flagged" table); visits; and alerts. The window can be 30, 90, 180 or 365 days.

Opening a record is written to the audit log, because it is personal health data.

Files involved

Only the files this feature uses.

```
backend/app/
├── api/routes/phc.py     GET /phc/citizens/{id}/record, _entry() (one timeline row)
├── api/views.py          checks_of(): the stored per-vital judgement of a flag
└── schemas/phc.py        RecordOut, RecordVital, TimelineEntry
frontend/src/
├── pages/phc/CitizenRecord.tsx          the page
├── components/app/ModelInsights.tsx     ScoreBreakdown, BasisTag
└── services/phc.ts                      getRecord(userId, days)
```

How the pieces connect

1. The page checks the id in the URL looks like a UUID, then calls `GET /phc/citizens/{id}/record?days=90`.
2. The backend confirms the citizen is in the catchment, loads the window's readings, alerts and visits, and writes an audit entry.
3. Each reading becomes a timeline entry with its severity, plain-language reason and stored model details.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/api/routes/phc.py` · citizen_record (start)

```
567  async def citizen_record(
568      user_id: uuid.UUID,
569      request: Request,
570      days: int = Query(90, ge=7, le=365),
571      worker: User = Depends(require_catchment_worker),
572      db: AsyncSession = Depends(get_session),
573  ) -> RecordOut:
574      citizen = await get_catchment_citizen(db, worker, user_id)
575      profile = await db.scalar(select(UserProfile).where(UserProfile.user_id == citizen.id))
576      end = now_utc()
577      start = end - timedelta(days=days)
578      in_window = and_(HealthRecord.timestamp >= start, HealthRecord.timestamp <= end)
579
580      readings_query = (
581          select(HealthRecord, ProcessedData)
582          .outerjoin(ProcessedData, ProcessedData.health_record_id == HealthRecord.id)
583          .where(HealthRecord.user_id == citizen.id, in_window)
584      )
```

What the code means

- `days: int = Query(90, ge=7, le=365)`: the window, 7 to 365 days.
- `and_(A, B)` combines two conditions; `in_window` is reused for several queries.
- The readings query joins scores the same way as the citizen's history page (guide 03).

2. `backend/app/api/routes/phc.py` · `_entry`

```python
def _entry(record: HealthRecord, processed: ProcessedData | None) -> TimelineEntry:
    flagged = bool(processed and processed.is_anomaly)
    return TimelineEntry(
        reading_id=record.id,
        processed_id=processed.id if processed else None,
        measured_at=record.timestamp,
        vitals=record.vitals,
        symptoms=record.symptoms,
        flagged=flagged,
        severity=severity_of(processed.anomaly_score) if flagged and processed else None,
        signal=describe_signal(processed.features, record.vitals) if flagged and processed else None,
        resolution=processed.resolution if processed else None,
        reviewed_at=processed.reviewed_at if processed else None,
        basis=processed.basis if processed else None,
        checks=checks_of(processed),
    )
```

What the code means

- Builds one timeline row. `processed.id if processed else None` handles readings without a score row.
- `describe_signal(...)` produces the sentence such as "Blood pressure 167/103 mmHg — well above the normal range".
- `checks_of(processed)` adds the model's stored per-vital working (next item).

3. `backend/app/api/views.py` · `checks_of`

```python
def checks_of(processed: ProcessedData | None) -> list[VitalCheck]:
    """The stored per-vital judgement of a score. Empty for rows scored before it was stored."""
    if processed is None or not processed.details:
        return []
    return [
        VitalCheck(**detail, key=key, contribution=processed.features.get(key, 0.0))
        for key, detail in processed.details.items()
    ]
```

What the code means

- `processed.details` is the JSON the model saved (guide 01). Old rows scored before it existed return an empty list.
- `VitalCheck(**detail, key=key, contribution=...)`: `**detail` spreads a dictionary into named arguments.

4. `frontend/src/pages/phc/CitizenRecord.tsx` · the submissions timeline

```tsx
              {r.timeline.map((t) => (
                <li key={t.reading_id} className="py-2 text-sm">
                  <div className="flex flex-wrap items-center gap-2">
                    <span className="text-ink">{fmtDateTime(t.measured_at)}</span>
                    {t.severity && <SeverityBadge severity={t.severity} />}
                    {t.flagged && <span className="text-xs text-ink-soft">{t.resolution ? t.resolution : "open"}</span>}
                    {t.flagged && <BasisTag basis={t.basis} />}
                  </div>
                  {t.signal && <p className="text-sev-high">{t.signal}</p>}
                  {t.flagged && t.checks.length > 0 && (
                    <details className="mt-1">
                      <summary className="cursor-pointer text-xs font-semibold text-brand">Why it was flagged</summary>
                      <div className="mt-2">
                        <ScoreBreakdown checks={t.checks} metrics={metrics} who="worker" />
                      </div>
                    </details>
                  )}
                  <p className="text-xs text-ink-soft">
                    {Object.entries(t.vitals).map(([k, v]) => `${metrics.get(k)?.label ?? k} ${v.toFixed(metrics.get(k)?.decimals ?? 0)}`).join(" · ")}
                    {t.symptoms.length > 0 && ` · ${t.symptoms.join(", ")}`}
                  </p>
                </li>
              ))}
            </ul>
```

```
205          )}
```

What the code means

- `r.timeline.map(...)` draws one list item per reading.
- `{t.flagged && <BasisTag basis={t.basis} />}` shows "vs their baseline" or "vs normal range" only on flagged rows.
- `<details>` / `<summary>` is a native HTML expand/collapse; the model's breakdown table sits inside it.
- `Object.entries(t.vitals)` turns {"heart_rate": 72} into pairs for display.

5. `frontend/src/components/app/ModelInsights.tsx` · BasisTag

```
67  export function BasisTag({ basis }: { basis: string | null }) {
68    if (!basis) return null;
69    const personal = basis === "personal_baseline";
70    return (
71      <span
72        className={cx("rounded-full px-2 py-0.5 text-xs font-medium", personal ? "bg-paper-raised text-brand"
              : "bg-paper-raised text-ink-soft")}
73        title={personal ? "Judged against this person's own usual readings" : "Judged against published normal
               ranges"}
74      >
75        {personal ? "vs their baseline" : "vs normal range"}
76      </span>
77    );
78  }
```

What the code means

- Returns `null` (draws nothing) for flags scored before the model stored its basis.
- `title=...` gives a tooltip explaining the tag.

Syntax used in this feature

Code	What it means
`and_(a, b)`	SQL AND of two conditions.
`X(**d)`	Pass a dictionary as named arguments.
`<details><summary>`	HTML expand/collapse without JavaScript.
`Object.entries(obj)`	[[key, value], ...] pairs of an object.